

Basic multi-threading principles

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

Sequential vs parallel execution

- Sequential processing -> operations happen one after another
- Parallel processing -> multiple operations happen simultaneously

How?

- Modern CPUs have multiple cores
- Thread: **the smallest unit of program execution**
- **Multi-threading** distributes work across these cores

Single-threading is like having a single-lane road where cars must follow one another. Multi-threading is like having multiple lanes where cars can travel simultaneously!

Using threads

- `Thread` class allows creating new execution paths
- Each `Thread` can execute independently

```
Runnable task = () -> {  
    System.out.println("Processing on thread: " +  
        Thread.currentThread().getName());  
};  
  
Thread thread = new Thread(task);  
thread.start();
```

```
Processing on thread: Thread-0
```

Working with multiple threads

```
List<Thread> threads = new ArrayList<Thread>();  
for (int i = 0; i < 4; i++) {  
    Thread thread = new Thread(() -> System.out.println("Processing data on Thread-" + i));  
    threads.add(thread);  
    thread.start();  
}  
  
for (Thread t : threads) {  
    t.join(); // Waits for all threads to complete  
}
```

```
// Processing data on Thread-0  
// Processing data on Thread-2  
// Processing data on Thread-1  
// Processing data on Thread-3
```

Parallel streams

- Streams - Java 8+ feature for simplified parallelism
- Automatically handles thread creation and management
- Two ways to create:
 - `collection.parallelStream()`
 - `Stream.of(...).parallel()`

Parallel streams example

```
// Sequential processing
List<Integer> result1 = new ArrayList<>();
for (int i = 0; i < numbers.size(); i++) {
    result1.add(numbers.get(i) * 2);
}

// Sequential processing with stream
List<Integer> result2 = numbers.stream()
    .map(n -> n * 2)
    .collect(Collectors.toList());

// Parallel processing with parallel stream
List<Integer> result3 = numbers.parallelStream()
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

When to use parallel processing

- CPU-intensive operations
- Independent data processing
- Large data collections
- Available CPU cores > 1
- Parallelization overhead may not be worth it for:
 - Small data sets
 - Simple operations

Summary

- `Thread` class for creating parallel execution paths
- Parallel streams for simplified collection processing
- Benefits depend on:
 - Workload type
 - Data size
 - Available cores

Let's practice!

OPTIMIZING CODE IN JAVA

Advanced threading patterns

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

Thread pool fundamentals

Thread creation is expensive!

Thread pools:

- Reuse existing threads
- Control number of concurrent threads
- Are managed through `ExecutorService` interface

Creating thread pools

```
// Fixed thread pool with 4 threads
```

```
ExecutorService fixedPool = Executors.newFixedThreadPool(4);
```

```
// Cached thread pool that grows as needed
```

```
ExecutorService cachedPool = Executors.newCachedThreadPool();
```

```
// Single-threaded executor
```

```
ExecutorService singleExecutor = Executors.newSingleThreadExecutor();
```

Submitting tasks

```
ExecutorService executor = Executors.newFixedThreadPool(4);

// Submit a task with no return value
executor.execute(() -> System.out.println("Simple task"));

// Submit a task with a return value (Callable)
Future<Integer> future = executor.submit(() -> {
    Thread.sleep(1000);
    return 42;
});

// Get result from Future (blocks until complete)
int result = future.get();
```

Shutting down executors

```
// Signal shutdown, but continue running existing tasks
executor.shutdown();

// Wait for termination (with timeout)
boolean terminated = executor.awaitTermination(5, TimeUnit.SECONDS);

// Force immediate shutdown, canceling running tasks
executor.shutdownNow();
```

The completable future

- `CompletableFuture`
- Part of Java's concurrency API since Java 8
- Modern approach to asynchronous programming
- Can be completed manually or via a `Function`
- Allows chaining operations with callbacks
- Works with or without explicit thread pools

Creating completable futures

```
// Run async with default executor
CompletableFuture<Void> runAsync =
    CompletableFuture.runAsync(() -> performTask());

// Supply async with custom executor
ExecutorService executor = Executors.newCachedThreadPool();
CompletableFuture<String> supplyAsync =
    CompletableFuture.supplyAsync(() -> fetchData(), executor);
```

Chaining operations

```
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> fetchUserData(userId))
    .thenApply(data -> extractUsername(data))
    .exceptionally(ex -> "Unknown user");

// Access to result when ready
future.thenAccept(result -> System.out.println(result));
```

Let's practice!

OPTIMIZING CODE IN JAVA

Caching strategies

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatos

Lead Engineer @Wealthyhood

What is caching?

Caching is like keeping our frequently used cooking ingredients on the kitchen counter instead of in the cupboard - it's faster to access, but we have limited counter space

We commonly cache:

- Database query results
- API responses
- Expensive calculations
- Other resource-intensive operations

In-memory caching in Java

```
// In-memory cache using HashMap
public class SimpleCache<K, V> {
    private final Map<K, V> cache = new HashMap<>();

    public V get(K key) {
        return cache.get(key);
    }

    public void put(K key, V value) {
        cache.put(key, value);
    }
}
```

Cache eviction policies

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- FIFO (First In, First Out)
- Time-based expiration
- And more!

Redis for distributed caching

What is Redis:

- In-memory data store/cache
- Excels as a distributed cache
- Supports many data structures
- Many Java client libraries - one example is Jedis



Using Redis with Jedis

```
import redis.clients.jedis.Jedis;

// Using Jedis client
Jedis jedis = new Jedis("localhost");
jedis.set("key", "value");
String value = jedis.get("key");
```

Additional features:

- Automatic expiration of cached items
- Cluster support

Implementing a time-based cache with Redis

```
public class RedisTimedCache {  
    private final Jedis jedis;  
  
    public RedisTimedCache(String host, int port) {  
        this.jedis = new Jedis(host, port);  
    }  
  
    public String get(String key) {  
        return jedis.get(key);  
    }  
  
    public void put(String key, String value, int timeToLiveSeconds) {  
        // Sets both the value and expiration time in seconds  
        jedis.setex(key, timeToLiveSeconds, value);  
    }  
}
```

Summary

- Caching stores computed results to avoid recalculation
- Effective for expensive operations and frequently accessed data
- Implement proper eviction policies to manage memory
- Consider distributed caching for multi-server applications (e.g. Redis with Jedis)

Let's practice!

OPTIMIZING CODE IN JAVA

Lazy initialization and singleton patterns

OPTIMIZING CODE IN JAVA

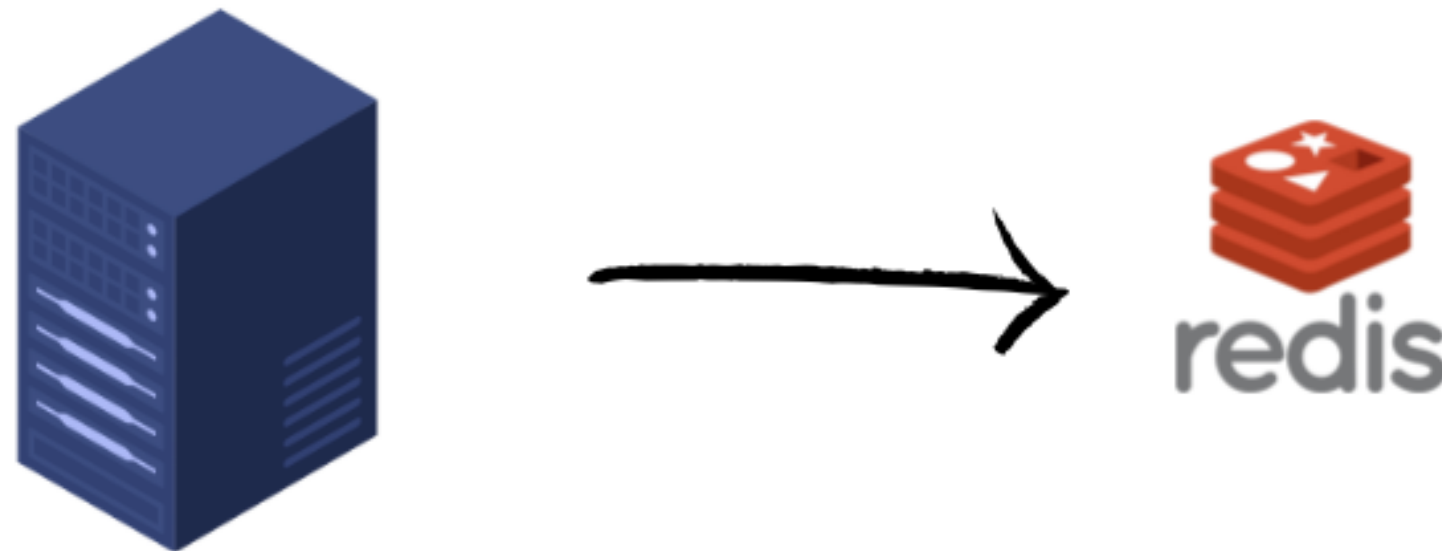


Pavlos Kosmetatos

Lead Engineer @Wealthyhood

Building a Redis cache client

- Imagine we are building a cache client that hits Redis
- Our Redis client is hosted at a third-party provider
- Setting up the connection to our client involves a network call - imagine this takes 500ms



A simple (eager) implementation

```
public class RedisCache {  
    // The client library we use to connect to Redis  
    private final RedisClient client;  
  
    public RedisCache() {  
        // The connection is setup inside the constructor  
        connection = new RedisClient("cache.mycompany.com");  
    }  
}
```

- Eager initialization of a Redis connection

A potential issue with this approach

This seems simple - and would work - but it has a potential issue

What if we don't need the Redis client at all?

- We wasted precious time at startup
- We established unnecessary connections

Lazy initialization

```
public class RedisCache {  
    private RedisClient client;  
  
    // Instead of setting up the connection in the constructor,  
    // we only set it up when someone needs to get the client.  
    public RedisClient getClient() {  
        if (connection == null) {  
            connection = new RedisClient("cache.mycompany.com");  
        }  
        return connection;  
    }  
}
```

Another issue with our approach

```
// UserService needs cache access
public class UserService {
    private RedisCache userCache = new RedisCache(); // First connection
}

// PaymentService also needs cache
public class PaymentService {
    private RedisCache paymentCache = new RedisCache(); // Second connection
}

// ...same for OrderService ...
```

The singleton pattern

```
public class RedisCache {  
    private static RedisCache instance;  
    private RedisClient client;  
  
    // The constructor is private so that we ensure we only  
    // create RedisCache inside this class  
    private RedisCache() {}  
  
    public static RedisCache getInstance()  
        // We only create a RedisCache if one does not already exist  
        if (instance == null) { instance = new RedisCache(); }  
        return instance;  
}  
  
// ... The rest is the same as before ...  
}
```

Let's practice!

OPTIMIZING CODE IN JAVA

Wrap-up

OPTIMIZING CODE IN JAVA



Pavlos Kosmetatatos
Lead Engineer @Wealthyhood

Chapter 1: Fundamentals of code performance

- Big-O Notation
 - Time Complexity
 - Space Complexity
- Data Structure Selection
 - HashSet
 - HashMap

Chapter 2: Measuring code performance

- `System.nanoTime()` for precise timing
- JVM Architecture
- Performance Metrics
 - `Runtime.getRuntime()` for memory tracking
 - `ThreadMXBean` for CPU analysis

Chapter 3: Improving code performance

- Multi-threading Fundamentals
 - Parallel streams
 - `ExecutorService`
- Caching Strategies
 - Jedis (Redis)
- Optimization Patterns
 - Singleton pattern implementation
 - Lazy vs eager initialization

Thank you!
OPTIMIZING CODE IN JAVA